Flex Pool Audit

Gnosis Ltd - Report by Côme du Crest

2025-07-11

Table of contents

- Table of contents
- Flex Pool Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med] AcrossFillTaker enforce too little give amount due to fees
  - [Med] maxWithdraw and maxRedeem discrepancy with ERC4626
  - [Low] Fusion swaps could be cancelled on source chain even after being filled
  - [Info] Users can profit from rebalance assets without balancing pools
  - [Info] Nonce in useUniswapPermit could be repeated
  - Optimisations and miscellaneous

Flex Pool Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within `swaps-io/flex-pool-contracts` as of commit `0xf01e565`.

Context

The project allows solvers to give assets to a pool on one chain and take the assets back from a matched pool on another chain in order to fulfill cross-chain transfers. Liquidity providers fill the pool and expect to receive a fee. A variable protocol fee is taken/given to the solver for increasing/decreasing the pool's imbalance.

The project integrates with various protocols such as Across, Circles' CCTP protocol, or 1Inch's Fusion to move assets from one pool to the other or to the end user.

Fusion swap

At first glance, it looks feasible to steal funds from the pool through Fusion swap cancellation. After looking at different approach, I believe it is not possible and write these notes as an argument why.

1Inch fusion protocol uses maker and taker addresses in the immutables of the escrow. The maker is the one emitting the swap intent. The taker is the solver that takes the maker's tokens on source and pay out the maker with its tokens on destination chain.

If the order is cancelled the escrowed tokens are reimbursed to the maker on source chain and to the taker on destination chain.

In `FusionGiver` the stored immutable uses `address(this)` as the taker but nothing is enforced. The maker is the address of the solver providing the funds.

In `FusionTaker` the taker is enforced to be `giveFusionGiver` in `_verifySrcImmutables()` to verify the give event emitted during escrow creation on source chain. However, it uses `address(this)` as the `immutable.taker` in `_composeDstImmutables()` when composing immutables. This means that `FusionTaker` will receive the pool assets back when an order is cancelled.

`FusionTaker` will be the taker on destination chain. Only `FusionTaker` will be able to cancel the escrow. The function `FusionTaker.cancelEscrow()` has the `returnPoolAsset` modifier

that will return the cancelled escrow pool tokens back to the pool immediately so the funds can not be stolen. Cancelling the escrow yields the control flow to the caller due to `trackNative` modifier sending the safety deposit received from the cancellation to caller. In that state, the `FusionTaker` holds assets but none of the assets can be stolen due to the `trackToken` modifiers on useful non-view functions.

Status

The report has been sent to the core developer.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Swaps.io (the “Company”), with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Med] AcrossFillTaker enforce too little give amount due to fees

Summary

AcrossFillTaker takes assets from the flex pool and ensures the input amount value is greater than the minimum amount of assets that should be given back to the pool. However, the input amount used by Across does not take into account Across fees which means that the give pool could receive less tokens than `minGiveAmount`.

Vulnerability Detail

The function that takes assets from the pool ensures that `inputAmount_ > minGiveAssets` and verifies that the event is correctly emitted with that input amount. It then calls `_fillRelay()` with the same `inputAmount_` value:

```
contract AcrossFillTaker ... {
    ...
    function takeToFillRelay(
        uint256 assets_,
        uint256 inputAmount_,
        uint256 outputAmount_,
        uint256 depositId_,
        uint32 quoteTimestamp_,
        uint32 fillDeadline_,
        uint32 exclusivityDeadline_,
        bytes32 depositor_,
        bytes32 recipient_,
        bytes32 exclusiveRelayer_,
        bytes calldata message_,
        bytes calldata depositProof_
    ) public override trackToken(poolAsset) {
        (uint256 takeAssets, uint256 minGiveAssets) = pool.take(assets_);

        _verifyTakeAssets(takeAssets, outputAmount_);  // @audit-info
            takeAssets > outputAmount
        _verifyGiveAssets(inputAmount_, minGiveAssets);  // @audit-info
            inputAmount > minGiveAssets
        _verifyGiveEvent(
            inputAmount_,
            ...
        );

        _fillRelay(
            inputAmount_,
            ...
        );
```

```
30      }
31  }
```

The function `_fillRelay()` fills the relay on `spokePool`:

```
1       function _fillRelay(
2           uint256 inputAmount_,
3           ...
4       ) private {
5           V3SpokePoolInterface.V3RelayData memory relayData =
                V3SpokePoolInterface.V3RelayData({
6               ...
7               inputAmount:         inputAmount_,                    // uint256  //
                    @audit-info inputAmount - across fee sent to givePool
8               ...
9           });
10          spokePool.fillRelay(
11              relayData,           // V3RelayData relayData
12              giveChain,           // uint256     repaymentChainId
13              _address32(givePool) // bytes32     repaymentAddress
14          );
15      }
```

`spokePool.fillRelay()` takes a fee on `inputAmount_` as stated in its documentation:

> inputAmount: This amount, less a system fee, will be sent to the caller on their repayment chain of choice as a refund following an optimistic challenge window in the HubPool.

Impact

The receiving pool will receive less than `minGiveAssets` tokens as Across fees will be paid from the `inputAmount_` resulting in a loss of funds for the protocol (or give pool liquidity providers).

Code Snippets

https://github.com/swaps-io/flex-pool-contracts/blob/f01e565a2f2e6f107c20d76cd2644a055ea7acb1/contracts/taker/across/AcrossFillTaker.sol

https://github.com/across-protocol/contracts/blob/087f220902e8fc4a452d2fa1c7fa9cda40a2d7d8/contracts/SpokePool.sol#L951

Recommendation

I did not see a clear way to set a limit on the fee paid to make sure `minGiveAssets` is lower than `inputAmount_ - maxFee`. Across messages could be used to execute logic verifying the received amount on give side.

[Med] maxWithdraw() and maxRedeem() discrepancy with ERC4626

Summary

The functions `flexPool.maxWithdraw()` and `flexPool.maxRedeem()` inherited from `ERC4626` no longer follow ERC4626 since there could be no asset in the vault.

Vulnerability Detail

According to ERC4626, the value returned by `maxWithdraw()` should not cause a revert when passed to `withdraw()` and should the highest accepted value:

> MUST return the maximum amount of assets that could be transferred from owner through withdraw and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).
>
> MUST factor in both global and user-specific limits, like if withdrawals are entirely disabled (even temporarily) it MUST return 0.

On the pool, the function `maxWithdraw()` will simply return the amount of shares held by the owner converted to assets:

```
1     function maxWithdraw(address owner) public view virtual returns (uint256) {
2         return _convertToAssets(balanceOf(owner), Math.Rounding.Floor);
3     }
4
5     function _convertToAssets(uint256 shares, Math.Rounding rounding) internal
          view virtual returns (uint256) {
6         return shares.mulDiv(totalAssets() + 1, totalSupply() + 10 **
              _decimalsOffset(), rounding);
7     }
8
9     function totalAssets() public view override(ERC4626, IERC4626) returns (
          uint256) {
10        return _totalAssets;
11    }  // @audit-info total assets deposited in the vault
```

If this value is used in the `withdraw()` function, it may fail if the pool is not balanced and most of its assets left the pool.

Similarly for `maxRedeem()`, as stated in ERC4626 it should return a value that would not cause a revert when passed to `redeem()`:

> MUST return the maximum amount of shares that could be transferred from owner through redeem and not cause a revert, which MUST NOT be higher than the actual maximum that would

> be accepted (it should underestimate if necessary).
>
> MUST factor in both global and user-specific limits, like if redemption is entirely disabled (even temporarily) it MUST return 0.

However, `maxRedeem()` currently simply returns the number of shares held by the owner:

```
    function maxRedeem(address owner) public view virtual returns (uint256) {
        return balanceOf(owner);
    }
```

This amount of shares may not be redeem if the pool is not balanced.

Impact

Discrepancy with ERC4626 may cause unexpected behaviour for users or other contracts building on top of the flex pool.

References

https://eips.ethereum.org/EIPS/eip-4626

Recommendation

Implement custom `maxRedeem()` and `maxWithdraw()` functions that take into account the currently available amount of assets held in the pool.

[Low] Fusion swaps could be cancelled on source chain even after being filled

Summary

If no monitoring is in place to withdraw the escrows on `FusionGiver` on a timely manner, escrows can be publicly cancelled on source chain to return the assets to the original taker even after being withdrawn on destination chain.

This means that the underlying pool will not receive any assets on source chain while assets are spent on destination chain.

Vulnerability Detail

There is no on-chain system in place to enforce that when assets are withdrawn on destination chain they can no longer be cancelled on source chain or that they need to be withdrawn on source chain before being withdrawn on destination chain.

Cancelling the source escrow will send back the assets to the maker of the order:

```
contract EscrowSrc is Escrow, IEscrowSrc {
    ...
    function _cancel(Immutables calldata immutables) internal
        onlyValidImmutables(immutables) {
        IERC20(immutables.token.get()).safeTransfer(immutables.maker.get(),
            immutables.amount);
        _ethTransfer(msg.sender, immutables.safetyDeposit);
        emit EscrowCancelled();
    }
    ...
}
```

Impact

If systems are not in place to ensure `FusionGiver.withdraw()` is called in a timely manner, pools could lose funds.

Code Snippets

https://etherscan.io/address/0xa7bcb4eac8964306f9e3764f67db6a7af6ddf99a#code

Recommendation

Acknowledge the issue or emit an event in `FusionGiver.withdrawEscrow()` that is verified on `FusionTaker.withdrawEscrow()` to ensure one is called before the other.

[Info] Users can profit from rebalance assets without balancing pools

Summary

If a pool has rebalance assets, a user can initiate a take on that pool to profit from the rebalance asset (e.g. through `FusionTaker` or `AcrossFillTaker`). The operation initiating the swap could be cancelled returning the assets to the pool but not increasing the `rebalanceAssets`, the surplus being sent to the taker.

Moreover, there is no consideration in place as to whether the corresponding pool on the other chain will become more balanced as a result of the operation.

The user could swap funds through each pools having `rebalanceAssets` to profit from `rebalanceAssets` without rebalancing any of the pool.

Vulnerability Detail

The `FlexPool.take()` function incentivize bringing balance to the pool by sending more tokens than asked for by the `assets_` argument when the pool has too much assets:

```
1     function take(uint256 assets_) public override returns (uint256 takeAssets,
          uint256 minGiveAssets) {
2         address tuner_ = tuner[_msgSender()];
3         require(tuner_ != address(0), NoTuner(_msgSender()));
4
5         (uint256 protocolAssets, int256 rebalanceAssets_) = ITuner(tuner_).tune
              (assets_);
6
7         minGiveAssets = assets_;
8         if (protocolAssets != 0) {
9             minGiveAssets += protocolAssets;
10            _totalAssets += protocolAssets;
11        }  // @audit-ok protocolAssets are given to the give pool and added in
              this pool
12
13        takeAssets = assets_;
14        if (rebalanceAssets_ > 0) {
15            minGiveAssets += uint256(rebalanceAssets_);
16            rebalanceAssets += uint256(rebalanceAssets_);  // @audit-ok
                  rebalanceAssets are given to the give pool and added in this
                  pool
17        } else {
18            takeAssets += uint256(-rebalanceAssets_);
19            rebalanceAssets -= uint256(-rebalanceAssets_);
20        }
21
22        SafeERC20.safeTransfer(IERC20(asset()), _msgSender(), takeAssets);
23        _verifyAssets();
24        emit Take(_msgSender(), assets_, protocolAssets, rebalanceAssets_);
```

```
25         }
```

If the operation that initiated the take is cancelled and the assets are returned to the pool, the `rebalanceAssets` value will not be increased and there will be no incentivization left.

Impact

Fragile incentivization mechanism to rebalance the pool.

Code Snippets

https://github.com/swaps-io/flex-pool-contracts/blob/f01e565a2f2e6f107c20d76cd2644a055ea7acb1/contracts/pool/FlexPool.sol#L81-L105

Recommendation

Incentivize pool rebalancing on the give side instead of the take side. I.e. if a pool does not have enough assets it should receive less and if a pool has too much assets it should receive more.

Alternatively, acknowledge the issue.

[Info] Nonce in useUniswapPermit() could be repeated

Summary

The function `AssetPermitter.useUniswapPermit()` computes a nonce to be used by uniswap Permit2 contract but the nonce is likely to be computed twice to the same value.

Vulnerability Detail

The function `useUniswapPermit()` computes the nonce from the encoding of asset address, from address, amount, deadline, and address of current contract. For a same user most of these values would be repeated.

```
    function useUniswapPermit(address from_, uint256 amount_, uint256 deadline_
        , bytes calldata signature_) external {
        uint256 nonce = uint256(keccak256(abi.encodePacked(permitAsset, from_,
            amount_, deadline_, address(this))));
        try IPermit2(PERMIT2).permitTransferFrom(
            PermitTransferFrom({
                permitted: TokenPermissions({
                    token: address(permitAsset),
                    amount: amount_
                }),
                nonce: nonce,  // @audit nonce could be repeated
                deadline: deadline_
            }),
            SignatureTransferDetails({
                to: address(this),
                requestedAmount: amount_
            }),
            from_,
            signature_
        ) {} catch {}
    }
```

This nonce is later consumed by Permit2:

```
    function _permitTransferFrom(
        PermitTransferFrom memory permit,
        SignatureTransferDetails calldata transferDetails,
        address owner,
        bytes32 dataHash,
        bytes calldata signature
    ) private {
        ...

        _useUnorderedNonce(owner, permit.nonce);

        ...
    }
```

```
14
15      function _useUnorderedNonce(address from, uint256 nonce) internal {
16          (uint256 wordPos, uint256 bitPos) = bitmapPositions(nonce);
17          uint256 bit = 1 << bitPos;
18          uint256 flipped = nonceBitmap[from][wordPos] ^= bit;
19
20          if (flipped & bit == 0) revert InvalidNonce();
21      }
```

If the contract attempt to use the same nonce twice, the permit will not apply.

Impact

If the same address attempts to use the uniswap permit2 feature with the same amount and deadline, the permit will silently fail and following interactions that need the transferred tokens will fail.

Code Snippets

https://github.com/swaps-io/flex-pool-contracts/blob/f01e565a2f2e6f107c20d76cd2644a055ea7acb1/contracts/permit/AssetPermitter.sol#L50

https://github.com/Uniswap/permit2/blob/cc56ad0f3439c502c246fc5cfcc3db92bb8b7219/src/SignatureTransfer.sol#L51

Recommendation

Use a different mechanism to compute the nonce that is more likely to be unique for example by having the user provide a nonce.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

TransferReliefGiver enables siphoning pool rebalance assets

The contract `TransferReliefGiver` calls an extra function `_takeReliefSurplus()` after giving assets to the flex pool. This function pretends giving relief to the pool by calling `tuner.setExtraReliefAssets(assets_)` with an inflated `assets_` value. Then it takes 0 asset from the pool to benefit from the rebalance assets when the pool is above equilibrium (i.e. pool has more assets that it should).

I do not understand the goal of this function and to me it seems to just allow to siphon the provisioned `rebalanceAssets` stocked in the pool.

My guess would be that the line `assets_ -= Math.min(assets_, uint256(equilibrium));` is incorrect and the subtraction `-=` should be replaced by a strict assignation `=`.

```
    function _takeReliefSurplus(uint256 assets_) private {
        int256 equilibrium = pool.equilibriumAssets();
        if (equilibrium > 0) {  // @audit-info pool has too much assets
            assets_ -= Math.min(assets_, uint256(equilibrium));  // @audit
                probably incorrect
        }
        if (assets_ == 0) {
            return;
        }

        tuner.setExtraReliefAssets(assets_);
        (uint256 protocolAssets, int256 rebalanceAssets) = tuner.tune(0);
        if (int256(protocolAssets) >= -rebalanceAssets) {
            tuner.setExtraReliefAssets(0);
            return;
        }

        (uint256 takeAssets, uint256 minGiveAssets) = pool.take(0);
        tuner.setExtraReliefAssets(0);

        SafeERC20.safeTransfer(poolAsset, address(pool), minGiveAssets);  //
            @audit-info only protocol assets
        SafeERC20.safeTransfer(poolAsset, msg.sender, takeAssets -
            minGiveAssets);  // @audit-info surplus of equilibrium - protocol
            assets
    }
```

_takeReliefSurplus() should use `tuneRelief()` instead of `tune()` to save gas

`tune()` just calls `tuneRelief()` with additional storage writes and checks that will be done anyway when the pool calls the tuner through `pool.take(0)`.

```
    function _takeReliefSurplus(uint256 assets_) private {
        int256 equilibrium = pool.equilibriumAssets();
        if (equilibrium > 0) {
            assets_ -= Math.min(assets_, uint256(equilibrium));
        }
        if (assets_ == 0) {
            return;
        }

        tuner.setExtraReliefAssets(assets_);
        (uint256 protocolAssets, int256 rebalanceAssets) = tuner.tune(0);  //
            @audit use `tuner.tuneRelief()` instead for gas savings
        if (int256(protocolAssets) >= -rebalanceAssets) {
            tuner.setExtraReliefAssets(0);
            return;
        }

        (uint256 takeAssets, uint256 minGiveAssets) = pool.take(0);
        tuner.setExtraReliefAssets(0);

        SafeERC20.safeTransfer(poolAsset, address(pool), minGiveAssets);
        SafeERC20.safeTransfer(poolAsset, msg.sender, takeAssets -
            minGiveAssets);
    }
```